



University of Houston Clear Lake

CENG 5133
CPU DESIGN

Muhammad Athar Waqas

Program Counter

The PC is a 4 bit-counter with an async, active-high reset and a clocked enable. When reset='1', the counter clears to 0000. On each rising edge of the clk, if load_pc='1' the PC increments by 1. If load_pc='0', it holds its current value. If it's at 15 and is instructed to increment, it wraps around and goes back to 0000. Internally it uses unsigned numbers for clean arithmetic and then the result is provided to pc_addr_out as a std_logic_vector.

Architecture

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pc is
    port(
        clk    : in std_logic;
        reset   : in std_logic;
        load_pc : in std_logic;
        pc_addr_out : out std_logic_vector(3 downto 0)
    );
end entity pc;

architecture Behavioral of pc is
    signal pc_reg : unsigned(3 downto 0) := (others => '0');
begin

    process(clk, reset)
    begin
        -- Reset = 1 Set pc_reg = 0
        if reset = '1' then
            pc_reg <= (others => '0');

        -- Load pc
        elsif rising_edge(clk) then
            if load_pc = '1' then
                pc_reg <= pc_reg + 1;    -- Increment pc
            end if;
        end if;
    end process;

    pc_addr_out <= std_logic_vector(pc_reg);

end Behavioral;
```

Test Bench

The PC testbench incorporates the 4-bit Program Counter, generates a free-running clock signal every 20 ns clock, and then drives reset and load_pc to exercise the behaviors. First we hold the reset = '1' so that the PC's output stays at 0000. Then after releasing the reset, it sets the load_pc = '1' and the clock runs for several cycles. The PC should increment one by one on every single rising edge clock. After this, we set load_pc = '0' and this should hold its current value. Midway we assert the reset signal to demonstrate that the asynchronous clear back to 0000.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity pc_tb is
end pc_tb;

architecture behavior of pc_tb is
    signal clk, load_pc, reset : std_logic;
    signal pc_addr_out : std_logic_vector(3 downto 0);
begin

    -- linking the entity work.pc
    UUT : entity work.pc
    port map(
        clk => clk,
        reset => reset,
        load_pc => load_pc,
        pc_addr_out => pc_addr_out
    );

    clock : process
    begin
        while TRUE loop
            clk <= '0';
            wait for 10 ns;
            clk <= '1';
            wait for 10 ns;
        end loop;
    end process;

    stimulus : process
    begin
        reset <= '1';
        load_pc <= '0';
        wait for 40 ns; -- Covers two rising edges
```

-- Release reset

reset <= '0';

wait for 10 ns;

-- First increment: hold load_pc high for full cycle

load_pc <= '1';

wait for 20 ns;

-- Second increment

load_pc <= '1';

wait for 20 ns;

-- Hold low

load_pc <= '0';

wait for 40 ns;

-- Reset again

reset <= '1';

wait for 20 ns;

reset <= '0';

wait for 20 ns;

-- Final increment

load_pc <= '1';

wait for 20 ns;

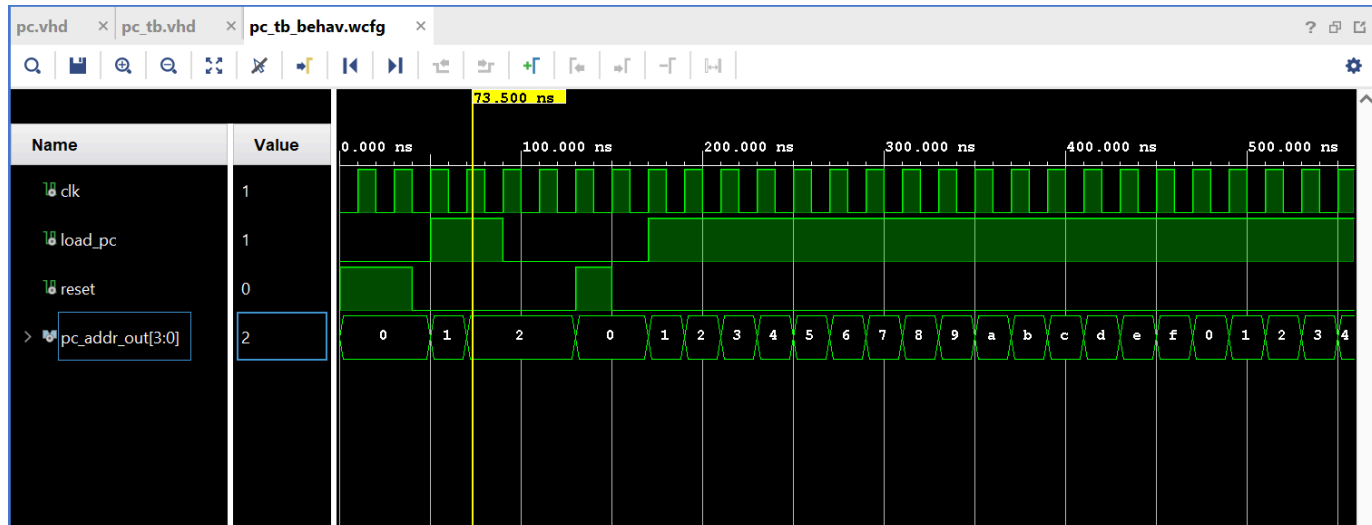
wait for 20 ns;

wait;

end process;

end behavior;

Wave Form



From the waveform, we can see that at start, `reset='1'` holds the `pc_addr_out='0'` despite clock activity. Once the reset is released, enabling `load_pc` allows the counter to increment step-by-step ($0 \rightarrow 1 \rightarrow 2$) on each clock cycle. When `load_pc` is low, the counter pauses and holds its current value. A mid-run reset pulse forces an immediate return to 0, which confirms the asynchronous operation is working. When the count reaches its maximum value of '1111' or 'F' or, it wraps around to '0000' which shows that it behaves as a 4-bit counter.

Register

The module is a customizable D-type register to store data and the bit-width can be customized. This is controlled by two main signals: asynchronous reset and synchronous load. When the reset = '1', the register immediately clears all data to zeroes. If the load = '1' the register captures the new input data, otherwise it holds the previous value. By default, the width is set to 16 bits, making the module ideal and suitable to use as an Instruction Register, pipeline and data registers.

Architecture

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cpu_reg_16 is
    generic (width : integer := 16);
    port (
        clk      : in  std_logic;
        reset    : in  std_logic;
        load     : in  std_logic;
        reg_in   : in  std_logic_vector(width-1 downto 0);
        reg_out  : out std_logic_vector(width-1 downto 0)
    );
end cpu_reg_16;

architecture Behavioral of cpu_reg_16 is
    signal q_reg : std_logic_vector(width-1 downto 0);
begin

    process(clk, reset)
    begin
        -- Reset reg
        if reset = '1' then
            q_reg <= (others => '0');
        -- Load reg, pass input to output
        elsif rising_edge(clk) then
            if load = '1' then
                q_reg <= reg_in;
            end if;
        end if;
    end process;

    reg_out <= q_reg;

end Behavioral;
```

TestBench

The testbench is designed to verify the 16-bit register using a continuous 20 ns clock. The simulation applies a sequence of specific test vectors –0x1234, 0x7425, and 0x2444 while toggling the load enable signal (ld). The register captures reg_in only when ld='1' at a rising edge and retains its value when ld='0'. This confirms that the register correctly captures new data only when enabled on a rising clock edge, and holds the previous value when disabled. The clock process also generates brief asynchronous reset pulses (5 ns high, 5 ns low) between clock phases to demonstrate that the register clears to 0 immediately whenever reset='1', regardless of the clock.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity cpu_reg_16_tb is
end entity;

architecture behaviour of cpu_reg_16_tb is
    constant width : integer := 16;

    -- Signals to connect to the cpu_reg_16
    signal reg_out, reg_in : std_logic_vector (width-1 downto 0);
    signal ld, clk, reset : std_logic;

begin
    -- Linking the entity to entity work.cpu_reg_16
    UUT : entity work.cpu_reg_16
        generic map (width => width)
        port map(
            reg_in => reg_in,
            load => ld,
            clk => clk,
            reset => reset,
            reg_out => reg_out
        );

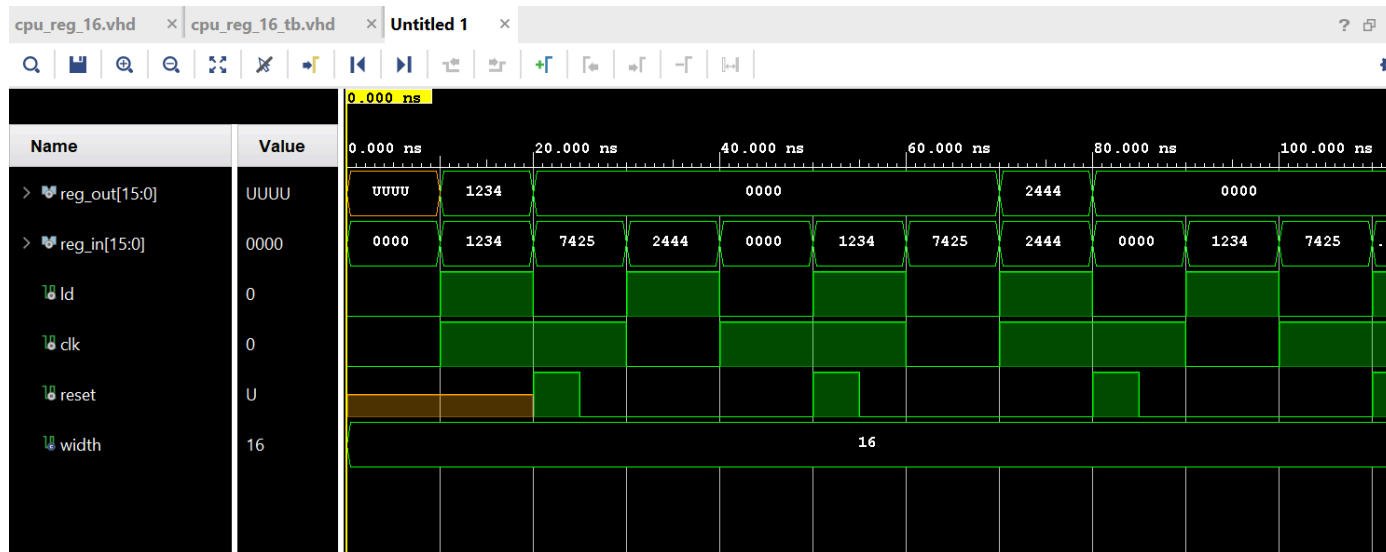
    clock : process
    begin
        while true loop
            clk <= '0';
            wait for 10 ns;
            clk <= '1';
            wait for 10 ns;
            reset <= '1';
            wait for 5 ns;
```

```
    reset <= '0';  
    wait for 5 ns;  
end loop;  
end process;
```

```
stimulus : process  
begin  
-- Initializing test cases  
ld <= '0';  
reg_in <= (others => '0');  
wait for 10 ns;  
ld <= '1';  
reg_in <= x"1234";  
wait for 10 ns;  
ld <= '0';  
reg_in <= x"7425";  
wait for 10 ns;  
ld <= '1';  
reg_in <= x"2444";  
wait for 10 ns;  
end process;
```

```
end behaviour;
```


Wave Form



The simulation begins with the output in an undefined state (UUUU). At ~20 ns, with ld=1 on a rising edge, the register loads 0x1234. A brief reset pulse immediately clears the output to 0000, showing reset overrides the clock. Later (~60 ns) it loads 0x2444, and another reset at ~80 ns clears it again to 0000. Whenever ld=0, input changes (e.g., 0x7425) are ignored and the output holds its previous value—confirming reset, load, and hold all work as intended. Basically, the waveform confirms all behaviors: reset clears instantly, ld=1 captures reg_in on a clock edge, and ld=0 makes the register hold.

MUX

Architecture

The module functions as a digital switch that selects between two width bit sources: an external `input_address` and the internal `prog_counter`. The selection is controlled by a single signal, `S`. When `S='0'`, the module passes the `input_address` through to the output. When `S='1'`, it switches to the `prog_counter`, allowing the processor to fetch instructions normally.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2 is
    generic ( width : integer := 4 ); -- size of each input
    port (
        input_address : in  STD_LOGIC_VECTOR(width-1 downto 0);
        prog_counter  : in  STD_LOGIC_VECTOR(width-1 downto 0);
        S              : in  STD_LOGIC;
        Y              : out STD_LOGIC_VECTOR(width-1 downto 0)
    );
end mux2;

architecture Behavioral of mux2 is
begin
    process(input_address, prog_counter, S)
    begin
        if S = '0' then
            Y <= input_address;    -- IM load phase
        elsif S = '1' then
            Y <= prog_counter;     -- normal execution (PC fetch)
        else
            Y <= (others => '0');  -- safety
        end if;
    end process;
end Behavioral;
```

TestBench

This specific testbench is designed to verify the instantaneous switching ability of the 4-bit multiplexer. We provide two distinct inputs—an `input_address` set to 0xA and a `prog_counter` set to 0xB. The simulation then toggles the select line `S`. When `S='0'`, the output `Y` should immediately show `input_address` (0xA). When `S='1'`, the output `Y` should immediately switch to `prog_counter` (0xB). The output changes can happen without any clock delay which shows that the MUX is operating correctly.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2_tb is
end mux2_tb;

architecture behaviour of mux2_tb is

-- Constants
constant width : INTEGER := 4;

-- Signals to connect to MUX2X
signal input_address, prog_counter : STD_LOGIC_VECTOR(width-1 DOWNT0 0);
signal S      : STD_LOGIC;
signal Y      : STD_LOGIC_VECTOR(width-1 DOWNT0 0);

begin

-- Linking the entity MUX4X
UUT: entity work.mux2
generic map (width => width)
port map (
    input_address => input_address,
    prog_counter => prog_counter,
    S => S,
    Y => Y
);

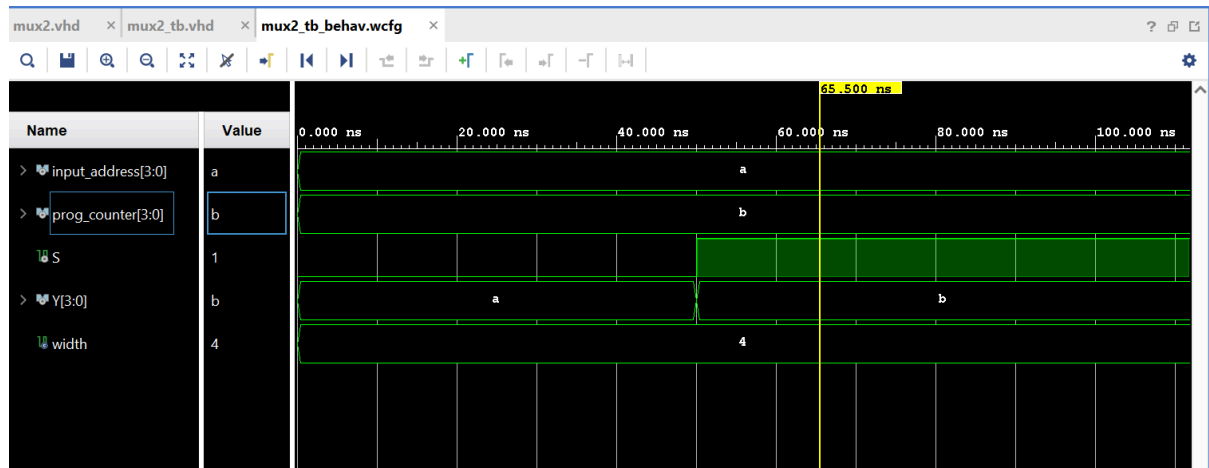
stimulus: process
begin
-- Initializing Input Test Cases
input_address <= X"A"; -- 1010101010101010
prog_counter <= X"B"; -- 1011101110111011

-- Select Lines
S <= '0'; wait for 50 ns;
S <= '1'; wait for 50 ns;
```

```
wait;  
end process;
```

```
end behaviour;
```

Wave Form



Input_address is fixed at A and prog_counter at B. From 0~50ns, S='0', so the output Y follows A. When S toggles to 1 (~50-60ns), Y immediately switches to B and stays there. There is no clock involved so this confirms that the MUX is combinational. Y=input_address when S=0, and Y=prog_counter when S=1 with width=4.

Instruction Register

Architecture

The Instruction Register serves as a 24-bit storage for the instruction currently being executed by the CPU. The IR's internal storage (instr) is managed by two distinct control signals: asynchronous active-high reset and a synchronous load. When reset='1', the internal register instr clears to all zeroes. On a rising clock edge, if load_ir='1' the IR latches instruction; if not it holds the current value. The output fields are as follows

- function_ir = instruction (bits 23-21)
- alu_opcode = instruction (bits 20-18)
- alu_mode = instruction (bit 17) – alu mode 0 or 1
- reg_index = instruction (bit 16) – register a or b
- data_address = instruction (bits 15-0) – memory address

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity cpu_IR_24 is

Port (

instruction : in std_logic_vector(23 downto 0);

clk : in std_logic;

load_ir : in std_logic;

reset : in std_logic;

function_ir : out std_logic_vector(23 downto 21);

alu_opcode : out std_logic_vector(20 downto 18);

alu_mode : out std_logic;

reg_index : out std_logic;

data_address : out std_logic_vector(15 downto 0)

);

end cpu_IR_24;

architecture Behavioral of cpu_IR_24 is

signal instr : std_logic_vector(23 downto 0) := (others => '0');

begin

process(clk, reset)

begin

if reset = '1' then

instr <= (others => '0');

elsif rising_edge(clk) then

if load_ir = '1' then

instr <= instruction;

end if;

end if;

```

end process;

-- Decode fields
function_ir <= instr(23 downto 21);
alu_opcode  <= instr(20 downto 18);
alu_mode    <= instr(17);
reg_index   <= instr(16);
data_address <= instr(15 downto 0);

end Behavioral;

```

TestBench

This testbench is set up to fully verify the operation of the 24-bit Instruction Register (IR) using a continuous 40 ns clock cycle and applies a reset followed by three instruction loads. First, reset='1' for 40 ns to clear the IR. After reset is done, the stimulus presents three distinct 24-bit instructions = 0x002424, then 0x101000, then 0x202233, each time asserting load_ir='1' which is long enough to span a rising edge. On each edge with load_ir=1, the IR latches the 24-bit word and the decoded outputs (function_ir, alu_opcode, alu_mode, reg_index, data_address) update to the proper bit slices; when load_ir=0, the fields hold their previous values.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity cpu_IR_24_tb is
end entity;

architecture behaviour of cpu_IR_24_tb is
    signal instruction : std_logic_vector(23 downto 0);
    signal clk, load_ir, reset : std_logic;
    signal alu_mode, reg_index : std_logic;
    signal function_ir : std_logic_vector(23 downto 21);
    signal alu_opcode : std_logic_vector(20 downto 18);
    signal data_address : std_logic_vector(15 downto 0);

begin

    -- Linking the signals to the entity work.cpu_IR_24
    UUT : entity work.cpu_IR_24
        port map(
            instruction => instruction,
            clk => clk,

```

```

    load_ir => load_ir,
    reset => reset,
    alu_mode => alu_mode,
    reg_index => reg_index,
    function_ir => function_ir,
    alu_opcode => alu_opcode,
    data_address => data_address
);

```

```

clock : process

```

```

begin

```

```

    clk <= '0';

```

```

    wait for 20 ns;

```

```

    clk <= '1';

```

```

    wait for 20 ns;

```

```

end process;

```

```

-- Loading the instructions

```

```

stimulus : process

```

```

begin

```

```

    reset <= '1';

```

```

    wait for 40 ns;

```

```

    reset <= '0';

```

```

    instruction <= x"002424"; load_ir <= '1';

```

```

    wait for 20 ns;

```

```

    load_ir <= '0';

```

```

    instruction <= x"101000"; load_ir <= '1';

```

```

    wait for 20 ns;

```

```

    load_ir <= '0';

```

```

    instruction <= x"202233"; load_ir <= '1';

```

```

    wait for 20 ns;

```

```

    load_ir <= '0';

```

```

    wait for 20 ns;

```

```

end process;

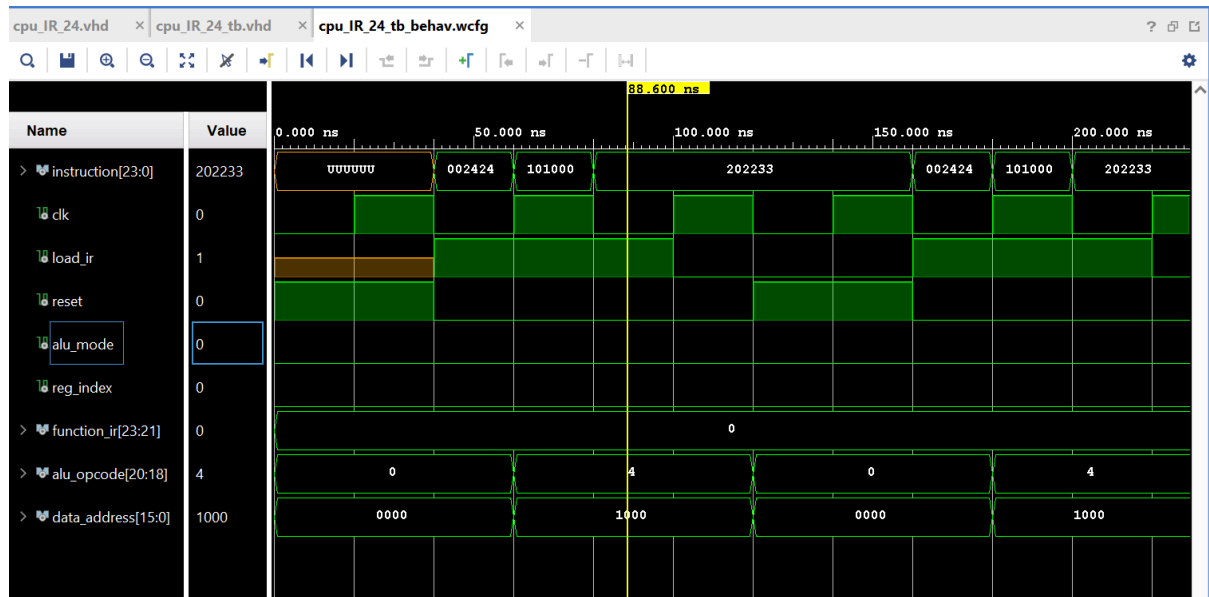
```

```

end behaviour;

```

Wave Form



At the beginning of the IR is reset, so all the decoded fields are 0. Each time `load_ir` is asserted across a rising clock edge, the 24-bit value on `instruction` is latched, and the field outputs immediately reflect the bit slices: `function_ir[23:21]`, `alu_opcode = instr[20:18]`, `alu_mode = instr[17]`, `reg_index = instr[16]`, and `data_address = instr[15:0]`. There are three loads 002424, 101000, and 202233. After each load, `alu_opcode` and `data_address` change to the values encoded in that instruction, and between loads the output holds steady. This shows that the IR clears on reset, captures the `load_ir`, and otherwise retains its contents.

Data Memory

The module acts as a small memory block, specifically 17-bits wide. On every rising edge of the clock, if the write enable signal `weDM='1'`, the memory location specified by the address is updated with the data provided by the `dm_in`. Reads are asynchronous and gated which means whenever `reDM='1'`, `dm_out` immediately reflects `mem[address]`; when `reDM='0'`, `dm_out` is all zeroes. Basically read shows the current word without waiting for a clock, while write only takes effect at the clock edge.

Architecture

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity data_mem is
    port (
        clk      : in std_logic;
        weDM     : in std_logic;
        reDM     : in std_logic;
        address  : in std_logic_vector(3 downto 0);
        dm_in    : in std_logic_vector(16 downto 0);
        dm_out   : out std_logic_vector(16 downto 0)
    );
end data_mem;

architecture Behavioral of data_mem is
    type memory_array is array (0 to 15) of std_logic_vector(16 downto 0);
    signal mem : memory_array := (others => (others => '0'));
    signal read_data : std_logic_vector(16 downto 0);
    signal write_data : std_logic_vector(16 downto 0);
begin

    -- Writing memory on rise clk
    process(clk)
    begin
        if rising_edge(clk) then
            if weDM = '1' then
                mem(to_integer(unsigned(address))) <= dm_in;
            end if;
        end if;
    end process;
```

```

-- Reading memory (not dependent on clk)
process(mem, address, reDM)
begin
    if reDM = '1' then
        read_data <= mem(to_integer(unsigned(address)));
    else
        read_data <= (others => '0');
    end if;
end process;

dm_out <= read_data;

end Behavioral;

```

TestBench

The testbench writes four 17-bits words to addresses 0000-0011 and then reads them back. A continuous 20 ns clock drives the component throughout the test. Writes are synchronous, so with weDM='1' each (address, dm_in pair is captured on the next rising edge. After the four writes, the TB switches to weDm='0', reDM='1' and steps every 20 ns because reads are asynchronous, dm_out updates immediately to the stored word for each address. When reDM='0', dm_out is forced to zeros.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.numeric_std.all;

entity data_mem_tb is

end data_mem_tb;

architecture Behavioral of data_mem_tb is
    signal weDM, reDM, clk : std_logic;
    signal dm_in, dm_out : std_logic_vector(16 downto 0);
    signal address : std_logic_vector(3 downto 0);
begin
    -- Linking the entity entity work.data_mem
    UUT : entity work.data_mem
    port map(

```

```
    weDM => weDM, reDM => reDM, clk => clk, dm_in => dm_in, dm_out => dm_out,  
    address => address  
);
```

```
clock : process  
begin  
while true loop  
    clk <= '0';  
    wait for 10 ns;  
    clk <= '1';  
    wait for 10 ns;  
end loop;  
end process;
```

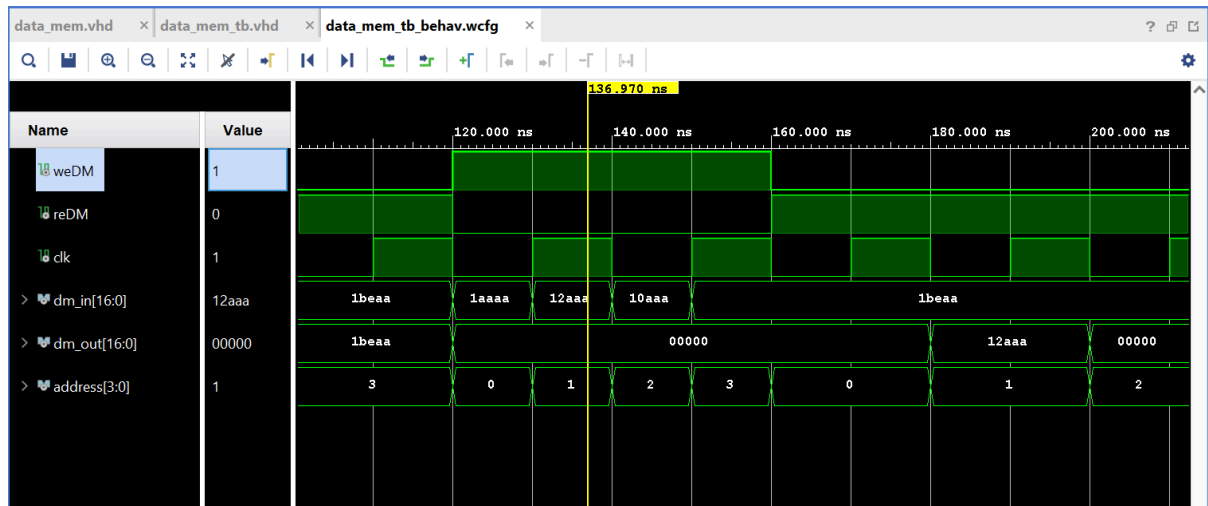
```
- -Inserting stimulus  
stimulus : process  
begin
```

```
-- Writing  
weDM <= '1'; reDM <= '0';  
address <= "0000"; dm_in <= "11010101010101010";  
wait for 10 ns;  
address <= "0001"; dm_in <= "10010101010101010";  
wait for 10 ns;  
address <= "0010"; dm_in <= "10000101010101010";  
wait for 10 ns;  
address <= "0011"; dm_in <= "11011111010101010";  
wait for 10 ns;
```

```
-- Reading  
weDM <= '0'; reDM <= '1';  
for i in 0 to 3 loop  
    address <= std_logic_vector(to_unsigned(i, 4));  
    wait for 20 ns;  
end loop;  
end process;
```

```
end Behavioral;
```

Wave Form



From ~120–160 ns the write-enable `weDM` is high, so on each rising clock edge the word on `dm_in` is stored at the current address (e.g., values like `beaa`, `1aaaa`, `12aaa`, `10aaa` get written as the address steps 0>1>2>3). During this write phase `reDM`=0, so `dm_out` is forced to 00000. After the write window, `weDM` goes low and the TB steps the address while `reDM` is asserted; because reads are asynchronous, `dm_out` immediately shows the stored value for each address. Basically writes are synchronous to the clock, while reads are immediate when `reDM`=1 and are zeroed when `reDM`=0.

Instruction Memory

Instruction memory is a 24-bit instruction memory (addresses 0..15). Writes are synchronous: on the rising edge of clk, if weIM='1' the word on input_instruction is stored at address. Read is combinational, instruction continuously reflects mem[address], so the output changes immediately when the address changes and no clock is needed. The array is initialized to zeroes.

Architecture

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity instr_mem is
  port (
    clk          : in std_logic;
    weIM         : in std_logic;
    input_instruction : in std_logic_vector(23 downto 0);
    address      : in std_logic_vector(3 downto 0);
    instruction   : out std_logic_vector(23 downto 0)
  );
end instr_mem;

architecture Behavioral of instr_mem is
  type memory_array is array (0 to 15) of std_logic_vector(23 downto 0);
  signal mem : memory_array := (others => (others => '0'));
begin

  process(clk)
  begin
    -- writing memory
    if rising_edge(clk) then
      if weIM = '1' then
        mem(to_integer(unsigned(address))) <= input_instruction;
      end if;
    end if;
  end process;
end process;
```

```
-- Reading memory
instruction <= mem(to_integer(unsigned(address)));
```

```
end Behavioral;
```

TestBench

The Instruction Memory testbench fills a 24 bit instruction memory, then reads it back. A 20 ns clock drives the DUT. When weIM='1', the testbench steps through the first 13 addresses (0 through 12) and presents a different 24-bit instruction (eg:- 000300, 000601, 40000...). Writes are asynchronous, each instruction is permanently saved into the memory only when the rising clock edge occurs. When weIM='0', the testbench cycles back through the addresses 0 through 12. Reads are combinational, the instruction output instantly updates to display the stored 24-bit instruction as soon as the address changes, confirming it does not need to wait for the clock. Basically, the test confirms that the memory can be loaded during clock edges and then successfully reads back the programmed instructions instantly whenever the address is selected.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

```
entity instr_mem_tb is
end instr_mem_tb;
```

```
architecture behavior of instr_mem_tb is
```

```
    signal clk          : std_logic := '0';
    signal weIM          : std_logic := '0';
    signal input_instruction : std_logic_vector(23 downto 0);
    signal address       : std_logic_vector(3 downto 0);
    signal instruction    : std_logic_vector(23 downto 0);
```

```
begin
```

```
    UUT: entity work.instr_mem
        port map (
            clk          => clk,
            weIM         => weIM,
            input_instruction => input_instruction,
            address       => address,
            instruction    => instruction
```

);

clk_process : process

begin

 while true loop

clk <= '0';

 wait for 10 ns;

clk <= '1';

 wait for 10 ns;

 end loop;

end process;

-- Inserting Stimulus

stimulus : process

begin

-- Writing stimulus

weIM <= '1';

address <= "0000"; *input_instruction* <= x"000300"; wait for 20 ns; -- Ld A 0x0003

address <= "0001"; *input_instruction* <= x"000601"; wait for 20 ns; -- Ld B 0x0006

address <= "0010"; *input_instruction* <= x"400000"; wait for 20 ns; -- ADD

address <= "0011"; *input_instruction* <= x"200001"; wait for 20 ns; -- Store C 0x0001

address <= "0100"; *input_instruction* <= x"420000"; wait for 20 ns; -- AND

address <= "0101"; *input_instruction* <= x"200002"; wait for 20 ns; -- Store C 0x0002

address <= "0110"; *input_instruction* <= x"600001"; wait for 20 ns; -- RdMem 0x0001

address <= "0111"; *input_instruction* <= x"000B01"; wait for 20 ns; -- Ld B 0x000B

address <= "1000"; *input_instruction* <= x"440000"; wait for 20 ns; -- Mult AB

address <= "1001"; *input_instruction* <= x"200003"; wait for 20 ns; -- Store C 0x0003

address <= "1010"; *input_instruction* <= x"600003"; wait for 20 ns; -- RdMem 0x0003

address <= "1011"; *input_instruction* <= x"E00000"; wait for 20 ns; -- Exit

address <= "1100"; *input_instruction* <= x"600000"; wait for 20 ns; -- RdMem 0x0000

(should not happen)

weIM <= '0';

-- Read back instructions

 for *i* in 0 to 12 loop

address <= std_logic_vector(to_unsigned(*i*, 4));

 wait for 20 ns;

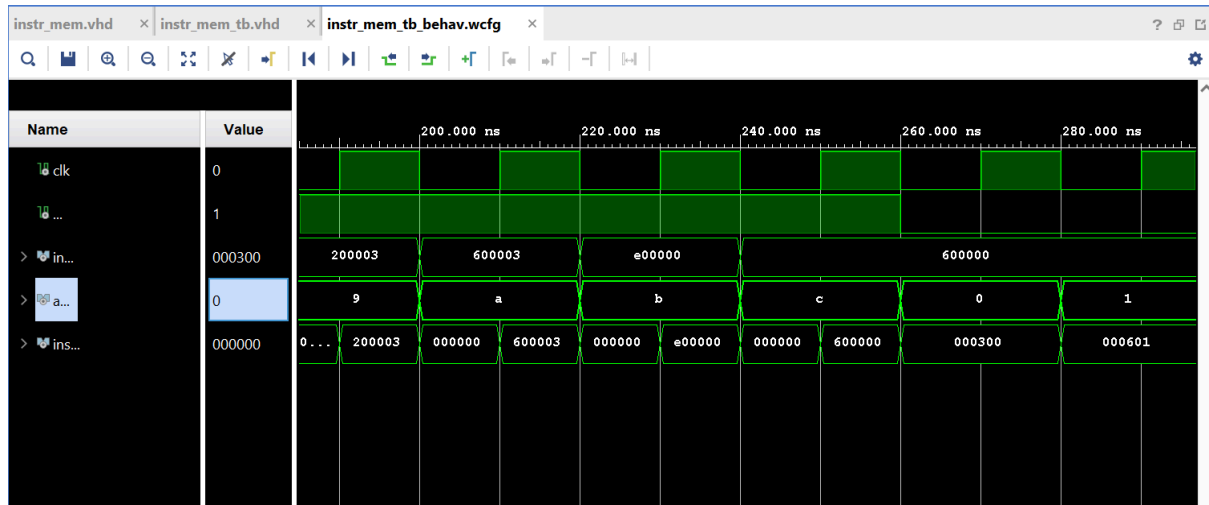
 end loop;

 wait for 10 ns;

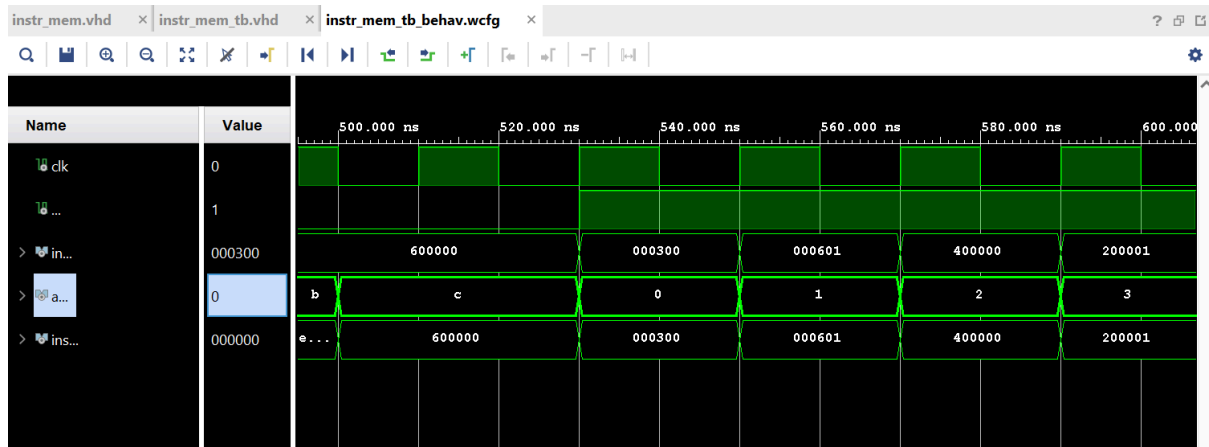
end process;

end behavior;

Wave Form



In this waveform we're in the readback phase with `weIM='0'`. The testbench steps the address every ~20 ns: 9 > a > b > c > 0 > 1. Because reads are combinational, the instruction output updates immediately when the address changes, showing the stored words 200003, 600003, E00000, 000000, 000300, and 000601. This confirms that writes occur on rising clock edges during the load phase (when `weIM='1'`), while fetches are instant during readback.



Here the `weIM='0'`. The reads are combinational so the instruction output updates immediately to the word stored at each location, the stored words are 600000, then 000300, 000601, 400000, and 200001 as the address advances. Basically it shows the behavior that the earlier writes happened on rising clock edges during the load phase; in readback, fetches appear instantly when the address changes.

CPU Controller

The CPU controller is basically the part that tells the whole processor what to do at each moment. It goes step-by-step through every instruction — fetch it, understand it, run it, and then move to the next one. Depending on the opcode, it decides whether we're loading A or B, doing an ALU operation, reading or writing memory, or stopping the CPU. It also sends the right signals to the ALU and updates the program counter after each instruction. Overall, it's like the coordinator that keeps the entire CPU running in order.

Architecture

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity cpu_controller is
  port (
    clk, reset, enable    : in std_logic;
    opcode_func           : in std_logic_vector(2 downto 0); -- bits 23-21
    alu_opcode_in         : in std_logic_vector(2 downto 0); -- bits 20-18
    alu_mode_in           : in std_logic;                    -- bit 17
    reg_index_in          : in std_logic;                    -- bit 16

    LoadA, LoadB, LoadC  : out std_logic;
    LoadIR, LoadPC       : out std_logic;
    WeDM, ReDM, Sel       : out std_logic;
    ALUMode               : out std_logic;
    ALUOpcode             : out std_logic_vector(2 downto 0);
    ex                    : out std_logic
  );
end entity;

architecture fsm of cpu_controller is

  type state_type is (IDLE, FETCH, DECODE, EXECUTE, INCREMENT_PC, HALT);
  signal state, next_state : state_type;

begin

  -- Synchronous execution
  process(clk, reset)
  begin
    if reset = '1' then
      state <= IDLE;
```

```

    elsif rising_edge(clk) then
        state <= next_state;
    end if;
end process;

```

-- Implementing FSM

```

process(state, enable, opcode_func, alu_opcode_in, alu_mode_in, reg_index_in)
begin

```

-- Initializing values

```

LoadA   <= '0';
LoadB   <= '0';
LoadC   <= '0';
LoadIR  <= '0';
LoadPC  <= '0';
WeDM    <= '0';
ReDM    <= '0';
Sel      <= enable;
ALUMode <= '0';
ALUOpcode <= (others => '0');
ex       <= '0';

```

```

next_state <= state;

```

case state is

```

when IDLE =>
    Sel <= '0';
    if enable = '1' then
        next_state <= FETCH;
    end if;

```

```

when FETCH =>
    LoadIR <= '1';
    next_state <= DECODE;

```

```

when DECODE =>
    case opcode_func is
        when "000" => next_state <= EXECUTE; -- Load A/B
        when "001" => next_state <= EXECUTE; -- Store C
        when "010" => next_state <= EXECUTE; -- ALU
        when "011" => next_state <= EXECUTE; -- RdMem
        when "111" => next_state <= HALT;   -- Exit
        when others => next_state <= FETCH;
    end case;

```

```

when EXECUTE =>
    case opcode_func is

```

```

when "000" =>  -- Load A or B
  if reg_index_in = '0' then
    LoadA <= '1';
  else
    LoadB <= '1';
  end if;

when "001" =>  -- Store C
  WeDM <= '1';

when "010" =>  -- ALU Op
  ALUMode  <= alu_mode_in;
  ALUOpcode <= alu_opcode_in;
  LoadC    <= '1';

when "011" =>  -- RdMem
  ReDM <= '1';

  when others => null;
end case;

-- Exit
if opcode_func = "111" then
  next_state <= HALT;
else
  next_state <= INCREMENT_PC;
end if;

when INCREMENT_PC =>
  LoadPC <= '1';
  next_state <= FETCH;

when HALT =>
  ex <= '1';
  LoadIR <= '0';
  LoadPC <= '0';
  WeDM  <= '0';
  ReDM  <= '0';
  next_state <= HALT;

end case;

end process;

end architecture;

```

TestBench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity cpu_controller_tb is
end cpu_controller_tb;

architecture tb of cpu_controller_tb is
    signal clk, reset, enable : std_logic := '0';
    signal opcode_func      : std_logic_vector(2 downto 0) := (others => '0');
    signal alu_opcode_in    : std_logic_vector(2 downto 0) := (others => '0');
    signal alu_mode_in      : std_logic := '0';
    signal reg_index_in     : std_logic := '0';

    signal LoadA, LoadB, LoadC, LoadIR, LoadPC : std_logic;
    signal WeDM, ReDM, Sel : std_logic;
    signal ALUMode : std_logic;
    signal ALUOpcode : std_logic_vector(2 downto 0);
    signal Halt : std_logic;
begin
    -- Linking the entity
    UUT: entity work.cpu_controller
    port map(
        clk => clk,
        reset => reset,
        enable => enable,
        opcode_func => opcode_func,
        alu_opcode_in => alu_opcode_in,
        alu_mode_in => alu_mode_in,
        reg_index_in => reg_index_in,
        LoadA => LoadA,
        LoadB => LoadB,
        LoadC => LoadC,
        LoadIR => LoadIR,
        LoadPC => LoadPC,
        WeDM => WeDM,
        ReDM => ReDM,
        Sel => Sel,
        ALUMode => ALUMode,
        ALUOpcode => ALUOpcode,
        ex => Halt
    );

    clock_process : process
```

```

begin
  while TRUE loop
    clk <= '0'; wait for 10 ns;
    clk <= '1'; wait for 10 ns;
  end loop;
end process;

-- Inserting Stimulus
stim_proc : process
begin
  -- Initial reset
  reset <= '1';
  enable <= '0';
  wait for 40 ns;
  reset <= '0';
  wait for 20 ns;

  -- Load
  wait for 40 ns;

  -- Execution
  enable <= '1';
  wait for 40 ns;

  -- Ld A instruction (opcode_func=000, reg_index=0)
  opcode_func <= "000";
  reg_index_in <= '0'; -- LoadA
  wait for 80 ns;

  -- Ld B instruction (opcode_func=000, reg_index=1)
  opcode_func <= "000";
  reg_index_in <= '1'; -- LoadB
  wait for 80 ns;

  -- ALU operation (opcode_func=010)
  opcode_func <= "010";
  alu_opcode_in <= "001";
  alu_mode_in <= '0';
  wait for 80 ns;

  -- Store C (opcode_func=001)
  opcode_func <= "001";
  wait for 80 ns;

  -- RdMem (opcode_func=011)
  opcode_func <= "011";
  wait for 80 ns;

```

```

-- Exit instruction (opcode_func=111)
opcode_func <= "111";
wait for 80 ns;

report "=== Test Complete ===";
wait;
end process;
end tb;

```

The test bench is used to test if all the stages of the controller are working as per expectation. It loads in the start and then enable is set to 1 that begins the execution. Different opcode_func represent different operations being performed by the controller.

WaveForm



After reset, the controller idles until enable goes high, then it cycles through fetch/decode/execute. For opcode_func=000, it loads the selected register: reg_index=0 → LoadA pulse; reg_index=1 → LoadB pulse. For the ALU op (010), it forwards ALUOpcode/ALUMode and issues a LoadC pulse to latch the ALU result. For Store (001), WeDM asserts to write Data Memory; for Load (011), ReDM asserts and LoadC latches the read value. LoadIR/LoadPC strobe at instruction boundaries. Finally, on Exit (111), the Halt output goes high and activity stops.

CPU Design - CPU_TOP (Project_1)

Architecture

The `cpu_top` is the top level description of the `cpu`. The small `cpu` wire together the smaller components together (instruction memory, registers, ALU, data memory, program counter, and controller) to form a complete processor. The inputs are as follows:- `clk`(clock), `reset`, `enable`, and instruction-memory programming signals (`weIM`, `InputInstruction`, `InputAddress`). The outputs are as follows:- `Cout` (ALU carry-out), `CPUout` (value of register C), and `DMout` (data memory output). **MUX** selects the source of the Instruction Memory address between the external `InputAddress` and the program counter `pc_addr_out`. Selection is controlled by 'Sel' and the selected address is provided as `address_out` to instruction memory. Instruction Memory (`instr_mem`) stores 24 bit instructions and when `weIM` is asserted, `InputInstruction` can be written into memory. The fetched 24 bit instruction is provided as `out_instruction`. `RegA` and `RegB` are 16 bit registers that load `data_address` when `LoadA` or `LoadB` is asserted. `RegC` is a 16 bit register that captures the ALU result (`ALUout`) when `LoadC` is asserted. The signals `A`, `B` and `C` hold the current register values. ALU (`cpu_alu_16`) receives operands `A` and `B` control (`alu_opcode_ctrl`, `alu_mode_ctrl`) from the controller and produces a 16 bit result `ALUout` and a carry out `Cout_int`. Data Memory (`data_mem`) is addressed by the low 4 bits of data address and it reads/writes 16 bit data. When writing the input, the concatenation is `Cout_int` & `C` (17 bits). The output is a 17 bit bus `DMdataOut` and the top 16 bits are exposed as `DMout`. The program counter (`pc`) maintains the address of the next instruction (`pc_addr_out`). The `cpu_controller` receives the decoded instruction fields and control inputs (`enable`, `reset`). Then Generates the control signals: `LoadA`, `LoadB`, `LoadC`, `LoadIR`, `LoadPC`, `WeDM`, `ReDM`, `Sel`. Following this `cpu_controller` forwards the ALU control signal (`alu_opcode_ctrl`, `alu_mode_ctrl`) and sequences datapath operations across cycles (fetch, decode, execute, memory, writeback).

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity cpu is
```

```
port (
```

```
    clk, reset, enable, weIM : in std_logic;
```

```
    InputInstruction          : in std_logic_vector(23 downto 0);
```

```
    InputAddress             : in std_logic_vector(3 downto 0);
```

```
    Cout                     : out std_logic;
```

```
    CPUout, DMout            : out std_logic_vector(15 downto 0)
```

```
);
```

```
end entity;
```

architecture Structural of cpu is

```
-- signal input_address  : std_logic_vector(3 downto 0) := (others => '0');
signal pc_addr_out      : std_logic_vector(3 downto 0);
signal address_out      : std_logic_vector(3 downto 0);

signal out_instruction  : std_logic_vector(23 downto 0);

-- IR fields
signal function_ir      : std_logic_vector(23 downto 21);
signal alu_opcode_ir    : std_logic_vector(2 downto 0);
signal alu_mode_ir      : std_logic;
signal reg_index        : std_logic;
signal data_address     : std_logic_vector(15 downto 0);

-- Controller outputs TO ALU
signal alu_opcode_ctrl  : std_logic_vector(2 downto 0);
signal alu_mode_ctrl    : std_logic;

-- Registers A, B, C
signal A, B, C          : std_logic_vector(15 downto 0);

-- Controller signals
signal LoadA, LoadB, LoadC : std_logic;
signal LoadIR, LoadPC      : std_logic;
signal WeDM, ReDM          : std_logic;
signal Sel                  : std_logic;

-- ALU
signal ALUout             : std_logic_vector(15 downto 0);
signal Cout_int           : std_logic;

-- Data Memory
signal DMdataOut          : std_logic_vector(16 downto 0);
```

begin

```
-- MUX (InputAddress vs PC)
MUX : entity work.mux2
port map(
    input_address => InputAddress, -- <-- drive from TOP PORT
    prog_counter  => pc_addr_out,
    S             => Sel,
    Y             => address_out
);
```

IM: entity work.instr_mem


```

port map(
    clk          => clk,
    weIM         => weIM,
    input_instruction => InputInstruction,
    address      => address_out,
    instruction   => out_instruction
);

```

IR: entity work.cpu_IR_24

```

port map(
    clk          => clk,
    load_ir      => LoadIR,
    reset        => reset,
    instruction  => out_instruction,
    function_ir  => function_ir,
    alu_opcode   => alu_opcode_ir,
    alu_mode     => alu_mode_ir,
    reg_index    => reg_index,
    data_address => data_address
);

```

RegA: entity work.cpu_reg_16

```

port map(
    clk    => clk,
    reset  => reset,
    load   => LoadA,
    reg_in => data_address,
    reg_out => A
);

```

RegB: entity work.cpu_reg_16

```

port map(
    clk    => clk,
    reset  => reset,
    load   => LoadB,
    reg_in => data_address,
    reg_out => B
);

```

RegC: entity work.cpu_reg_16

```

port map(
    clk    => clk,
    reset  => reset,
    load   => LoadC,
    reg_in => ALUout,
    reg_out => C
);

```

ALU: entity work.cpu_alu_16

```
port map(  
  a    => A,  
  b    => B,  
  opcode => alu_opcode_ctrl,  
  mode  => alu_mode_ctrl,  
  clk   => clk,  
  result => ALUout,  
  cout  => Cout_int  
);
```

DM: entity work.data_mem

```
port map(  
  clk    => clk,  
  weDM   => WeDM,  
  reDM   => ReDM,  
  address => data_address(3 downto 0),  
  dm_in  => Cout_int & C,  
  dm_out => DMdataOut  
);
```

PC: entity work.pc

```
port map(  
  clk    => clk,  
  reset  => reset,  
  load_pc => LoadPC,  
  pc_addr_out => pc_addr_out  
);
```

CTRL: entity work.cpu_controller

```
port map(  
  clk      => clk,  
  reset    => reset,  
  enable   => enable,  
  opcode_func => function_ir,  
  alu_opcode_in => alu_opcode_ir,  
  alu_mode_in  => alu_mode_ir,  
  reg_index_in => reg_index,  
  LoadA      => LoadA,  
  LoadB      => LoadB,  
  LoadC      => LoadC,  
  LoadIR     => LoadIR,  
  LoadPC     => LoadPC,  
  WeDM       => WeDM,  
  ReDM       => ReDM,  
  Sel        => Sel,  
  ALUMode    => alu_mode_ctrl,  
  ALUOpcode  => alu_opcode_ctrl,
```

```

        ex      => open
    );

    Cout  <= Cout_int;
    CPUout <= C;
    DMout <= DMdataOut(15 downto 0);

end architecture;

```

TestBench

The testbench is `cpu_tb` instantiates the `cpu` under unit test (UUT), which provides clock and stimulus signals, and simulates programming the instruction memory followed by executing the loaded program. The key stimulus signals are as follows:- `clk` (clock), `reset`, `enable` (starts execution) `weIM` (write-enable for instruction memory), `InputInstruction` (24-bit instruction), and `InputAddress` (4-bit instruction memory address). A free running clock is generated by the `clk_process`. It toggles every 10 ns, producing a 20 ns clock period. For the initial reset, the testbench asserts `reset` for 40ns, then resets it. During the reset the CPU is initialised. For the programming mode the testbench enters memory program mode by asserting `weIM='1'` and keeping `enable='0'` to prevent execution while writing instructions.

The testbench writes a sequence of 24 bit instructions into the instruction memory address 0000 through 1011. Each `InputInstruction` is formed by concatenating the instruction fields (function, ALU opcode, ALU mode, reg index) followed by a 16 bit immediate address. Each write waits 20 ns to emulate separate write cycles.

The instruction sequence are as follows:

- 0000: Load A with immediate 0003
- 0001: Load B with immediate 0006
- 0010: ADD A + B (ALU operation)
- 0011: Store C to memory address 0001
- 0100: Read memory from address 0001 (RdMEM)
- 0101: SUB A-B
- 0110: Store C to memory address 0002
- 0111: Read memory address 0001 (RdMem)
- 1000: MULT A * B (ALU opcode indicates multiply)
- 1001: Store C to memory address 0003
- 1010: Read memory from address 0003
- 1010: Exit halt instruction

After writing, `weIM` is deasserted and `InputAddress` is cleared. For switching to execution the testbench then sets `enable = '1'`, allowing the CPU controller to fetch and execute the loaded instructions from the program counter. Then the testbench waits for 2000 ns to let the CPU run through the program. A final wait; holds the simulation at the end.

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity cpu_tb is
end entity;

architecture behavior of cpu_tb is
    signal clk    : std_logic := '0';
    signal reset  : std_logic := '0';
    signal enable  : std_logic := '0';
    signal weIM   : std_logic := '0';

    signal InputInstruction : std_logic_vector(23 downto 0) := (others => '0');
    signal InputAddress    : std_logic_vector(3 downto 0) := (others => '0');

    signal Cout      : std_logic;
    signal CPUout    : std_logic_vector(15 downto 0);
    signal DMout     : std_logic_vector(15 downto 0);

begin
    -- Linking the entity
    UUT : entity work.cpu
    port map(
        clk => clk,
        reset => reset,
        enable => enable,
        weIM => weIM,
        InputInstruction => InputInstruction,
        InputAddress    => InputAddress,
        Cout => Cout,
        CPUout => CPUout,
        DMout => DMout
    );

    clk_process : process
    begin
        clk <= '0'; wait for 10 ns;
        clk <= '1'; wait for 10 ns;
    end process;

    -- Inserting Stimulus
    stimulus : process
    begin
        -- Reset
        reset <= '1'; enable <= '0'; weIM <= '0';
        wait for 40 ns;
    end process;

```

```

reset <= '0';
wait for 40 ns;

-- Loading
weIM <= '1'; enable <= '0';

-- Load A, 0003
InputAddress <= "0000";
InputInstruction <= "000" & "000" & "0" & "0" & x"0003";
wait for 20 ns;

-- Load B, 0006
InputAddress <= "0001";
InputInstruction <= "000" & "000" & "0" & "1" & x"0006";
wait for 20 ns;

-- ADD A + B
InputAddress <= "0010";
InputInstruction <= "010" & "000" & "0" & "0" & x"0000";
wait for 20 ns;

-- Store C
InputAddress <= "0011";
InputInstruction <= "001" & "000" & "0" & "0" & x"0001";
wait for 20 ns;

-- RdMem
InputAddress <= "0100";
InputInstruction <= "011" & "000" & "0" & "0" & x"0001";
wait for 20 ns;

-- SUB A - B
InputAddress <= "0101";
InputInstruction <= "010" & "001" & "0" & "0" & x"0000";
wait for 20 ns;

-- Store C
InputAddress <= "0110";
InputInstruction <= "001" & "000" & "0" & "0" & x"0002";
wait for 20 ns;

-- RdMem 2
InputAddress <= "0111";
InputInstruction <= "011" & "000" & "0" & "0" & x"0002";
wait for 20 ns;

-- MULT A * B (opcode 010, alu_opcode=010)
InputAddress <= "1000";

```

```

InputInstruction <= "010" & "010" & "0" & "0" & x"0000";
wait for 20 ns;

-- Store C
InputAddress <= "1001";
InputInstruction <= "001" & "000" & "0" & "0" & x"0003";
wait for 20 ns;

-- RdMem 3
InputAddress <= "1010";
InputInstruction <= "011" & "000" & "0" & "0" & x"0003";
wait for 20 ns;

-- Exit
InputAddress <= "1011";
InputInstruction <= "111" & "000" & "0" & "0" & x"0000";
wait for 20 ns;

weIM <= '0';
InputAddress <= (others => '0');

-- Execute
enable <= '1';
wait for 2000 ns;

wait;
end process;

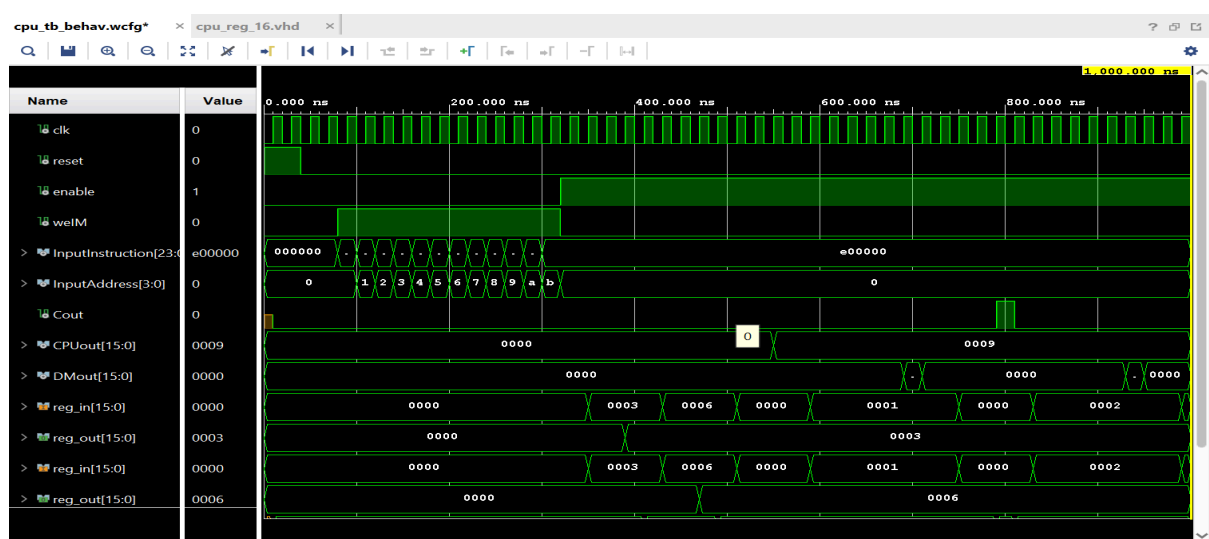
end architecture;

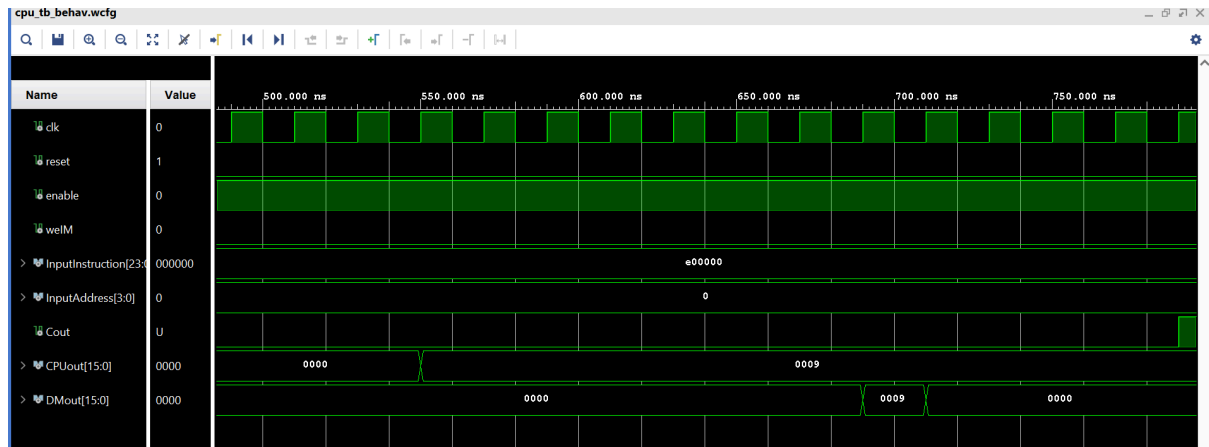
```

Wave Form



The clock is running the whole time. The CPU is idle enable=0, while weIM=1 to program the instruction memory. During the loading phase, InputAddress steps from (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b) and on each rising clock edge the 24-bit word on InputInstruction is written to that specific address. When the core is disabled, Cout, CPUout, and DMout stay 0000. After programming, weIM is asserted and enable is asserted, handing control to the CPU. The PC begins fetching from address 0, the IR latches the first instruction, and subsequent activity appears on CPUout/DMout as execution goes forward.





In this waveform, the clock runs with `weIM=0`, so we're no longer programming instruction memory. The CPU is enabled and executes. Around 640–700 ns a data-memory read occurs: the address goes to 0, `DMout` briefly shows 0009 (the word at that location), and `CPUout` echoes 0009 during the move/write-back stage before returning to 0000. `InputInstruction` remains 000000 because we're not loading code; the core is simply fetching from IM and operating internally. Between operations the datapath idles at zero, which is expected for this single-operation sequence.